Hilos JVM con Thread

creación, ejecución y ciclo de vida

En esta presentación aprenderemos a:

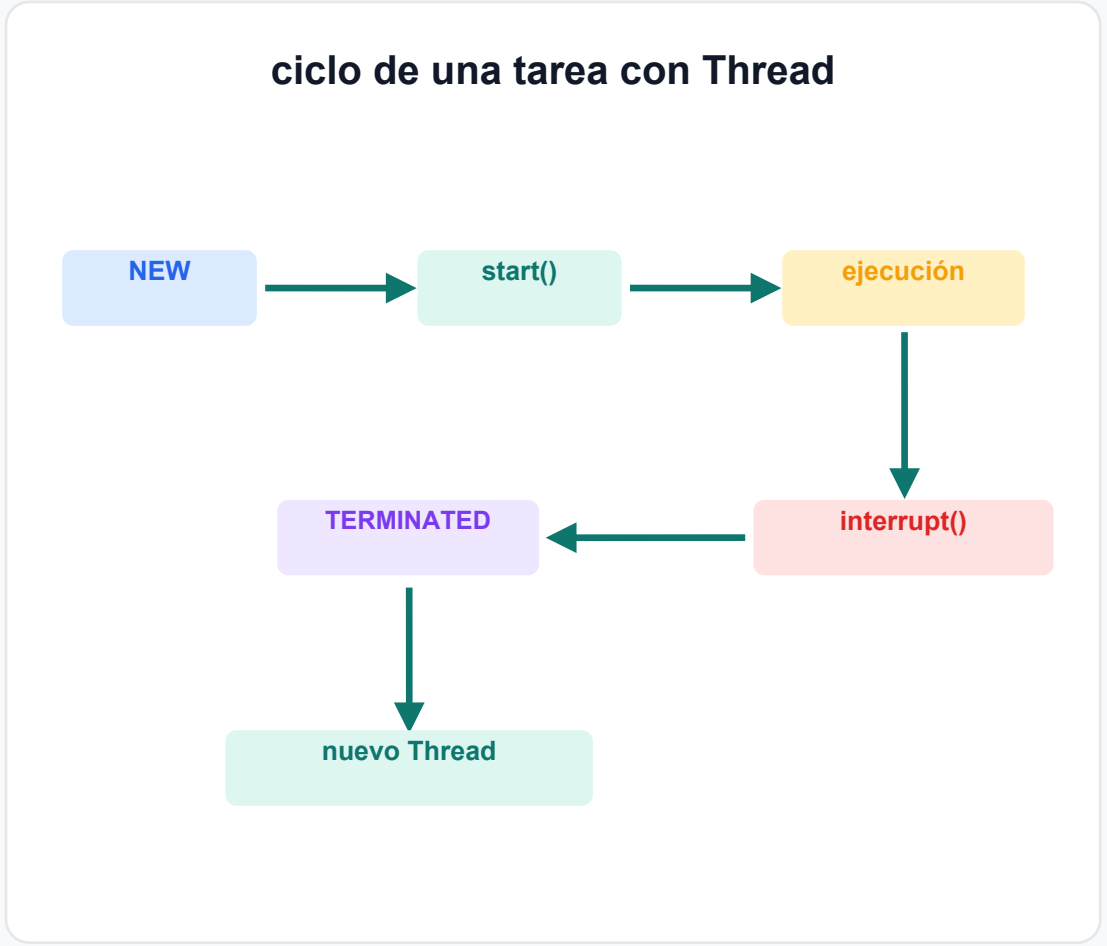
- crear y arrancar varios Thread
- distinguir run() de start()
- distinguir sleep() de join()
- observar estados de ejecución
- detener trabajo mediante interrupción cooperativa
- volver a ejecutar una tarea creando un nuevo Thread

Resultado final

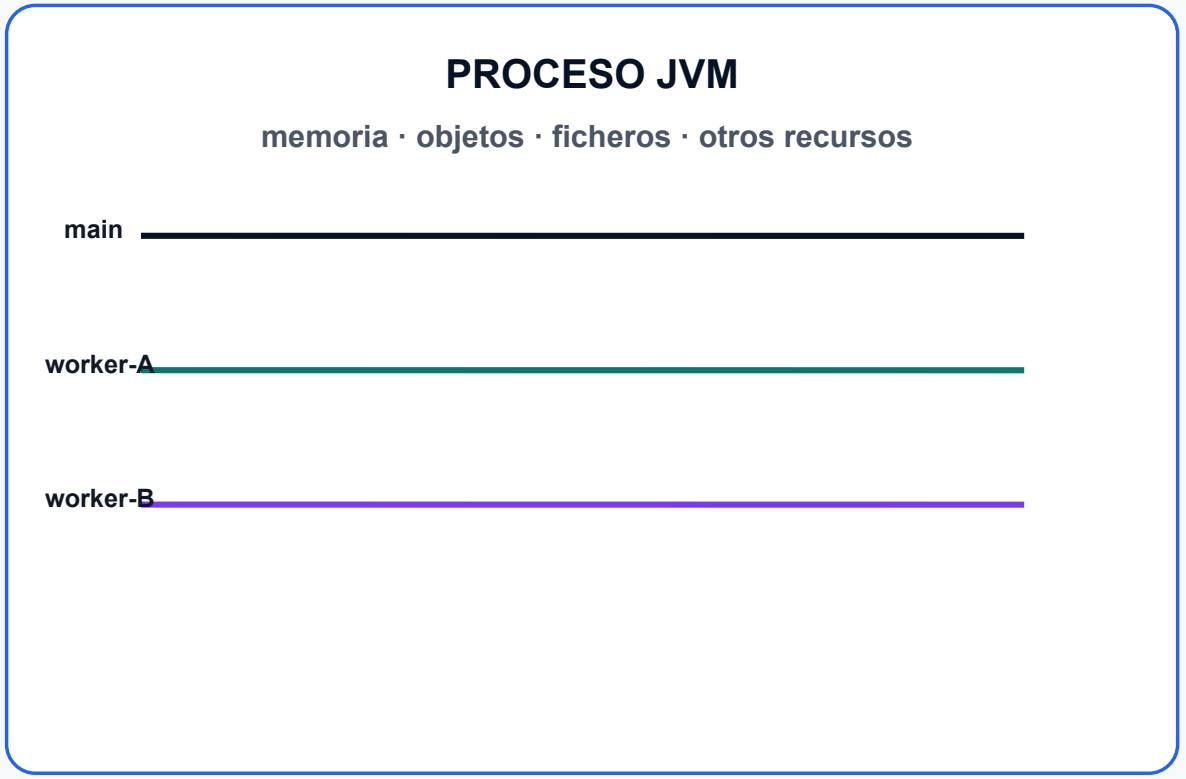
un segundero Kotlin/JVM capaz de:

arrancar → contar → detenerse → terminar → volver a arrancar con otro Thread

90 min · Kotlin/JVM · sin Android



Un proceso JVM, varios flujos de ejecución



Ya sabemos:

- un proceso contiene uno o más hilos;
- sus hilos utilizan recursos del mismo proceso;
- crear un proceso ≠ crear un hilo.

¿Por qué usar hilos aquí?

Buen candidato

- misma aplicación
- tareas que pueden progresar independientemente
- trabajo o esperas que pueden solaparse
- necesitan ejecutarse dentro del mismo proceso

No automáticamente

- más hilos = más complejidad
- una tarea pequeña o estrictamente dependiente puede seguir siendo secuencial

```
herramientas externas independientes → procesos
misma aplicación · varios flujos     → hilos
trabajo pequeño y dependiente        → secuencial
```

Antes de crear nada adicional ya existe un hilo

```
fun main() {
    println("main -> ${Thread.currentThread().name}")

    val worker = Thread {
        println(
            "trabajo -> ${Thread.currentThread().name}"
        )
    }

    worker.name = "worker-A"
    worker.start()
}
```

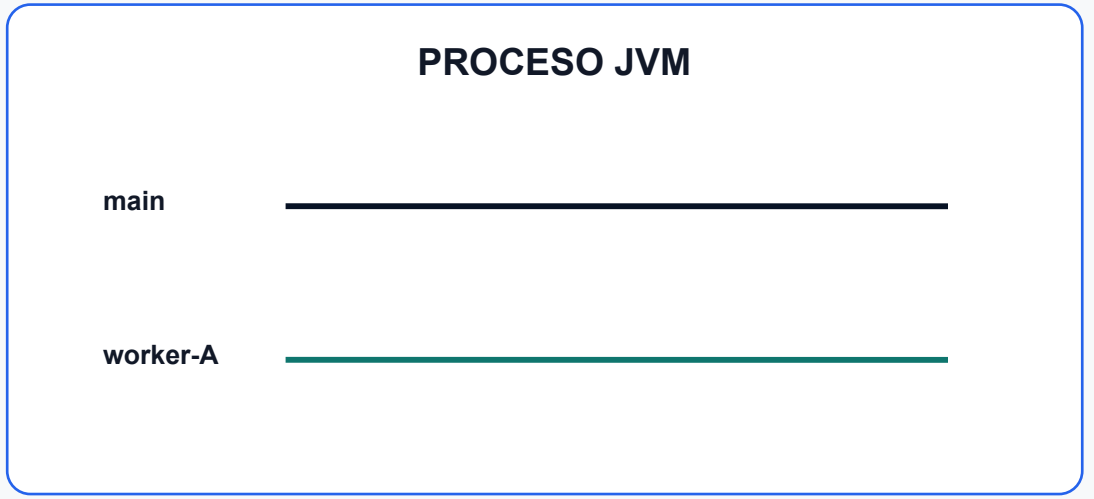
Salida:

```
main -> main
trabajo -> worker-A
```

`Thread.currentThread()`

→ devuelve una referencia al hilo que está ejecutando esa línea de código.

Después de start():



Dos formas mínimas de crear el trabajo

Dos formas que debemos saber leer y escribir

1. Subclase de Thread

```
class MiHilo : Thread() {
    override fun run() {
        println("Trabajo")
    }
}

val hiloA = MiHilo()
```

2. Thread con lambda

```
val hiloB = Thread {
    println("Trabajo")
}
```

`run()` → trabajo que ejecutará el hilo **`start()` → arranque del Thread**

Kotlin imprescindible: `: Thread()` → herencia `override fun run()` → redefinir `run()` `{ ... }` → lambda

No estamos estudiando Kotlin avanzado.

run() y start() no hacen lo mismo

El error más importante de esta presentación

```
fun main() {
    val worker = Thread {
        println(
            "Trabajo en ${Thread.currentThread().name}"
        )
    }

    worker.name = "worker-A"

    println("Llama ${Thread.currentThread().name}")
    worker.run()

    println("Ahora start()")
    worker.start()
}
```

Salida:

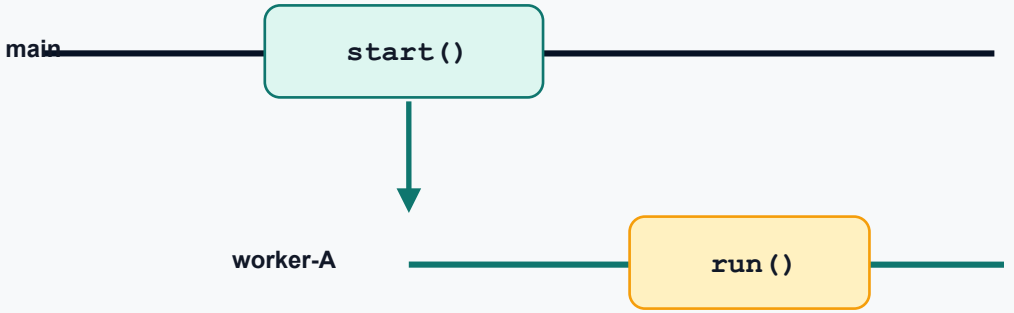
```
Llama main
Trabajo en main
Ahora start()
Trabajo en worker-A
```

run() directo



No arranca el objeto worker.

start()



Regla clave

En los Thread creados con las formas utilizadas en esta presentación:

`worker.run()` → llamada normal: ejecuta ese código el hilo llamante
`worker.start()` → arranca worker; worker ejecutará `run()`

Ahora sí: varios hilos

```
fun main() {
    val workerA = Thread {
        repeat(4) {
            println(
                "A$it · ${Thread.currentThread().name}"
            )
        }
    }

    val workerB = Thread {
        repeat(4) {
            println(
                "B$it · ${Thread.currentThread().name}"
            )
        }
    }

    workerA.name = "worker-A"
    workerB.name = "worker-B"

    workerA.start()
    workerB.start()
}
```

¿Cuál será el orden exacto?

Dentro de A:

`A0 → A1 → A2 → A3`

Dentro de B:

`B0 → B1 → B2 → B3`

Entre A y B: ¿?

Una ejecución observada no crea una garantía.

Micro-modificación

Añade: `worker-C`

con el mismo patrón y arráncalo también. Después identifica en la salida: A · B · C

Thread.sleep(): ¿qué hilo se detiene?

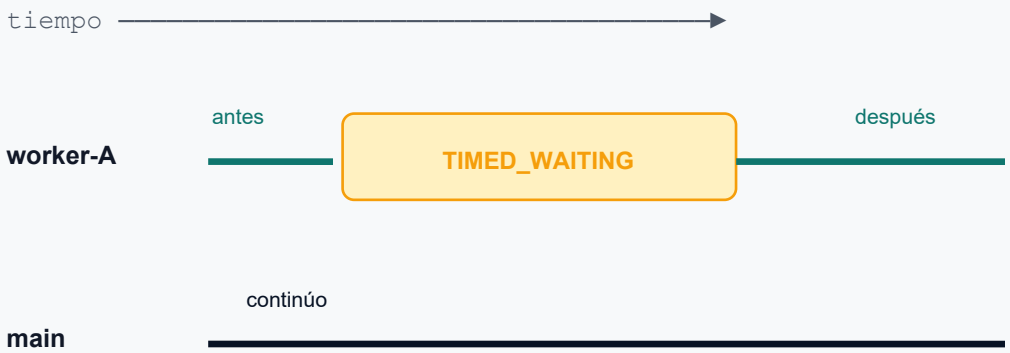
```
val worker = Thread {
    println("worker: antes")

    Thread.sleep(1000)

    println("worker: después")
}

worker.name = "worker-A"
worker.start()

println("main: continuó")
```



Regla clave

Thread.sleep(1000) hace esperar temporalmente al hilo que ejecuta esa llamada.

La duración y la reanudación efectiva dependen de temporizadores · planificación.
Una interrupción puede terminar la espera antes.

sleep() responde a:

¿esperar durante cuánto tiempo?

No responde a:

¿ha terminado otro hilo?

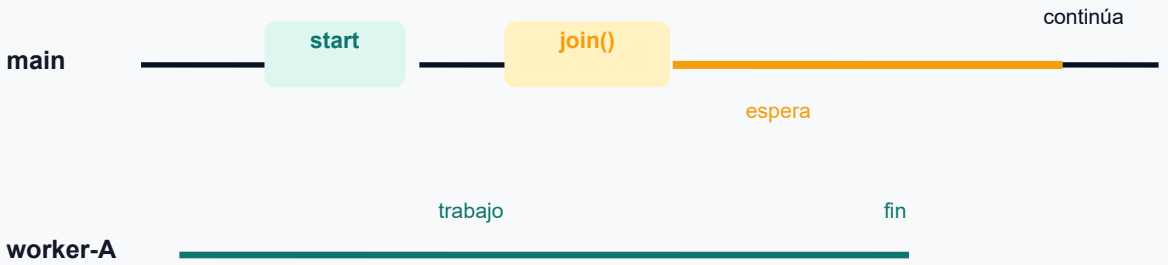
join(): esperar la terminación de otro hilo

```
val worker = Thread {
    Thread.sleep(2000)
    println("worker: fin")
}

worker.name = "worker-A"

worker.start()

println("main: voy a esperar")
worker.join()
println("main: worker ha terminado")
```



`worker.join()`

El hilo que ejecuta esa línea no continúa más allá de join() hasta que worker haya terminado.
Si worker ya había terminado, join() retorna inmediatamente.

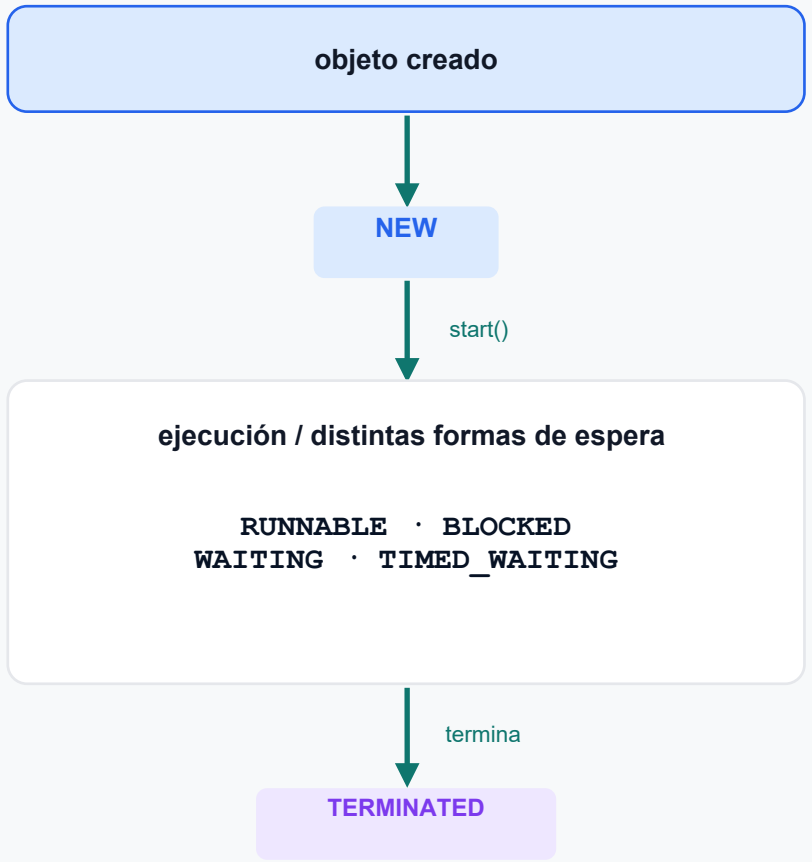
Dos problemas diferentes

```
Thread.sleep(1000)  → que transcurra tiempo
worker.join()       → que worker haya terminado
```

Puente con la presentación anterior

```
Process.waitFor() → terminación de un proceso
Thread.join()     → terminación de un hilo
```

Thread.State: no son los estados de proceso de la presentación anterior



Estados definidos por la JVM

`NEW`	creado, todavía no iniciado.
`RUNNABLE`	ejecutándose en la JVM; puede estar esperando recursos del SO, como procesador.
`BLOCKED`	esperando adquirir un monitor.
`WAITING`	esperando indefinidamente una acción.
`TIMED_WAITING`	esperando durante un tiempo determinado.
`TERMINATED`	ha finalizado.

Atención **Thread.State no contiene RUNNING**

estados de proceso de la anterior presentación ≠ Thread.State

Demostración de estados: NEW → TIMED_WAITING → TERMINATED

```
val worker = Thread {
    Thread.sleep(2000)
}

worker.name = "worker-A"

println(worker.state)
// NEW

worker.start()

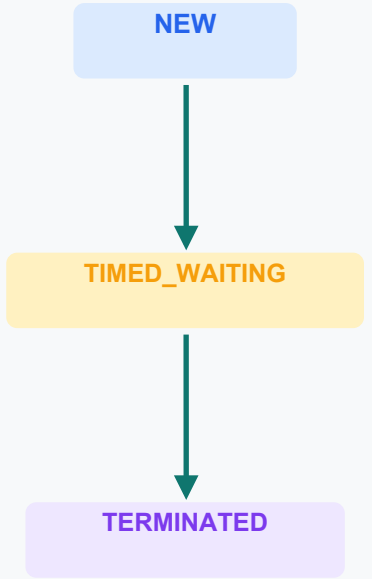
Thread.sleep(100)
// solo preparamos la observación

println(worker.state)
// normalmente TIMED_WAITING

worker.join()

println(worker.state)
// TERMINATED
```

Lo que comprobamos



antes de start()
→ NEW

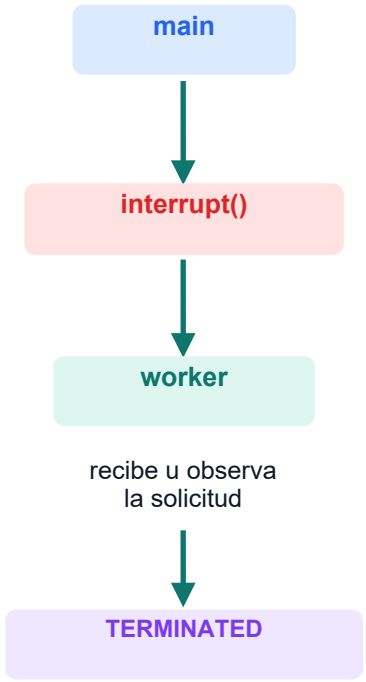
durante Thread.sleep()
→ TIMED_WAITING

después de terminar
→ TERMINATED

No confundas

“bloqueado” como concepto general ≠ Thread.State.BLOCKED
Durante Thread.sleep(): TIMED_WAITING

interrupt(): solicitar cancelación, no matar




```
class Worker : Thread("worker") {

    override fun run() {
        try {
            while (!isInterrupted) {
                println("trabajando")
                Thread.sleep(500)
            }
        } catch (e: InterruptedException) {
            println("interrumpido durante sleep")
        }

        println("fin")
    }
}
```

Control:

```
val worker = Worker()

worker.start()

Thread.sleep(1200)

worker.interrupt()
worker.join()
```

interrupt() no significa «kill»

ejecutando código → puede observar isInterrupted
operación interrumpible → InterruptedException
en este ejemplo → fin de run()

Un objeto Thread se arranca como máximo una vez

```
val worker = Thread {
    println("una ejecución")
}

worker.start()
worker.join()

println(worker.state)
// TERMINATED
```

Esto no es válido:

```
worker.start()
// ✕ ese objeto ya fue iniciado
```

Para ejecutar otra vez la tarea:

```
val worker2 = Thread {
    println("nueva ejecución")
}

worker2.start()
worker2.join()
```

Thread #1



✗ start() otra vez

Thread #2



reiniciar la tarea ≠ arrancar otra vez el mismo Thread
reiniciar la tarea = crear un nuevo Thread

Un segundo start() sobre el primero produce IllegalThreadStateException. No es necesario memorizar ese nombre.

Prioridad: existe, pero no controla la corrección

Con los Thread que utilizamos en esta presentación:

```
val worker = Thread {
    // trabajo
}

println(worker.priority)

worker.priority =
    Thread.NORM_PRIORITY + 1

println(worker.priority)
```

La prioridad:

- puede consultarse
- puede modificarse dentro de su rango
- expresa una prioridad relativa
- no proporciona una garantía de planificación

NO GARANTIZA

qué hilo se ejecuta primero
qué hilo termina primero
un intercalado determinado
la corrección del programa

Nunca utilices prioridad como mecanismo de coordinación.

APIs que no usaremos ahora para detener o pausar:

`Thread.stop()` `Thread.suspend()` `Thread.resume()`

Microprácticа · Segundero: objetivo y clase

Comportamiento elegido

```
0 aparece inmediatamente
~1 s → 1
~1 s → 2
~1 s → 3
...
```

Objetivo

- se ejecute en su propio Thread
- espere aproximadamente un segundo entre valores
- pueda detenerse cooperativamente
- confirme su finalización
- pueda volver a ejecutarse mediante un nuevo objeto

Pregunta

¿Podemos volver a hacer start() sobre el primer Segundero terminado?

```
class Segundero : Thread("segundero") {

    override fun run() {
        var segundos = 0

        try {
            while (true) {
                println(segundos)

                // TODO 1
                // esperar 1 segundo

                // TODO 2
                // sumar 1
            }
        } catch (e: InterruptedException) {
            println("Segundero detenido")
        }
    }
}
```

La diapositiva siguiente completa el control desde main.

Microprácticа · Segundero: control desde main

```
fun main() {
    var segundero = Segundero()

    // TODO 3: arrancar

    Thread.sleep(3500)
    // solo dejamos transcurrir tiempo

    // TODO 4: solicitar parada

    // TODO 5: esperar la finalización

    // TODO 6:
    // crear OTRO Segundero y arrancarlo
}
```

Después del reinicio:

deja pasar ~2,5 s
detén la segunda ejecución
espera su finalización

No utilizar

`Variable running compartida`
`synchronized`
`Android · Compose`

Recordatorio

Thread.sleep(3500) en main
solo deja pasar tiempo.
La finalización se comprueba con join().

Solución · Segundero

```
class Segundero : Thread("segundero") {

    override fun run() {
        var segundos = 0

        try {
            while (true) {
                println(
                    "$segundos · " +
                    Thread.currentThread().name
                )

                Thread.sleep(1000)
                segundos++
            }
        } catch (e: InterruptedException) {
            println("Segundero detenido")
        }
    }
}
```

Qué representa cada parte

`segundos`	→ dato local de esta ejecución
`run()`	→ trabajo
`println(segundos)`	→ 0 se muestra inmediatamente
`Thread.sleep(1000)`	→ espera temporal del segundero
`segundos++`	→ avanza después del intervalo
`InterruptedException`	→ la espera ha sido interrumpida
`fin de run()`	→ TERMINATED

Todavía no necesitamos estado mutable compartido.

Control desde main: primera ejecución

```
fun main() {
    var segundero = Segundero()

    segundero.start()

    Thread.sleep(3500)
    // main solo deja pasar tiempo

    segundero.interrupt()
    segundero.join()

    println(
        "1: vivo=${segundero.isAlive}, " +
        "estado=${segundero.state}"
    )

    // continuará con una nueva ejecución
}
```

Primera ejecución

```
start()
   ↓
0 · segundero
1 · segundero
2 · segundero
3 · segundero
   ↓
interrupt()
   ↓
join()
   ↓
vivo=false · TERMINATED
```

Muy importante

Thread.sleep(3500) en main
solo deja transcurrir tiempo para observar.

segundero.join()
confirma que el trabajador ha terminado

Control desde main: volver a ejecutar la tarea

```
fun main() {
    // ... primera ejecución ya terminada

    segundero = Segundero()
    segundero.start()

    Thread.sleep(2500)

    segundero.interrupt()
    segundero.join()

    println(
        "2: vivo=${segundero.isAlive}, " +
        "estado=${segundero.state}"
    )
}
```

Reinicio

nuevo Segundero()
↓
start()

No hacemos

TERMINATED → NEW
en el mismo objeto

Segunda ejecución

vuelve a empezar en 0
se detiene con interrupt()
se confirma con join()

No dormimos main para mantener vivo al trabajador.

No basta con que «parezca funcionar»

1. ¿Qué hilos existen?

```
main
worker-A · worker-B · worker-C
segundero
```

2. ¿Qué hilo ejecuta cada parte?

```
Thread.currentThread().name
```

3. ¿Qué hace cada operación?

```
start()     → arranca
sleep()     → espera tiempo
join()      → espera finalización
interrupt() → solicita interrupción
```

4. ¿Ha terminado?

```
worker.isAlive
worker.state
```

5. ¿Qué orden está garantizado?

```
orden establecido por el programa
≠
orden observado una vez
```

6. ¿Por qué utilizamos un hilo?

«El segundero es una tarea de la misma aplicación que puede progresar independientemente. Se ejecuta en un Thread, se cancela mediante interrupción y main espera su finalización con join().»

Ejecuta · observa · explica

Ya sabemos crear hilos; ahora aparece una nueva pregunta

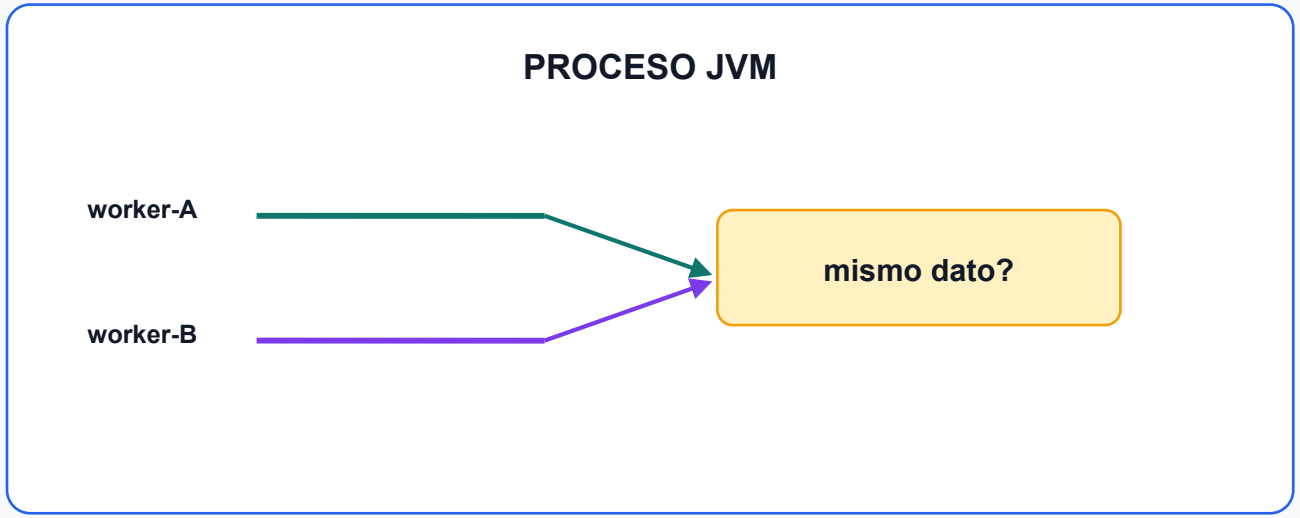
Lo que ya sabemos

```
crear Thread
      ↓
definir run()
      ↓
   start()
      ↓
 varios hilos
      ↓
sleep() / join()
      ↓
observar estados
      ↓
  interrupt()
      ↓
  TERMINATED
      ↓
nuevo Thread si queremos repetir
```

También sabemos:

- identificar el hilo actual
- usar nombres para diagnosticar
- no convertir un orden observado en garantía
- no utilizar prioridad para coordinar

Pero los hilos pertenecen al mismo proceso



Pregunta para la próxima presentación:

Dos hilos pueden estar perfectamente escritos por separado.
Si ambos modifican el mismo dato, ¿basta con eso para que el resultado conjunto sea correcto?